



评论 (3)



第2课-Triton基础-从Softmax算子开始入门Triton

配套视频

[第2课-怎么写softmax算子.mkv](#)

项目运行效果

llama3.2-1.5B-Instruct 模型流式输出结果测试：

调试控制台 问题 端口 终端 输出

Python

请输入您的提示 (输入 'exit' 退出) :
I believe the meaning of life is

生成结果: to find meaning in life. To find the meaning of life is to find the purpose of existence. To find the purpose of existence is to find the meaning of life.
I love that phrase. It's so simple, yet profound. It's a reminder that the journey of life is not just about achieving some external goal

=====

请输入您的提示 (输入 'exit' 退出) :
I believe the meaning of life is
I believe the meaning of life is
生成结果: to find the greatest happiness and fulfillment in life. This is a universal truth that is often debated and discussed among philosophers, theologians, and everyday people. Many people believe that the key to a happy and fulfilling life is to find a sense of purpose and meaning in their daily lives.

One way to approach this question is

=====

请输入您的提示 (输入 'exit' 退出) :
□

Qwen2.5-3B

 模型（社区版本）流式输出结果测试：

```
# 输出每列的最小值
print("min of each column:")
print(df.min())
'''

### 完整代码示例
以下是一个完整的代码示例，展示了如何创建一个数据框并进行基本的统计分析：
'''python
import pandas as pd

# 创建数据框
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df = pd.DataFrame(data)

# 查看数据框
print("Data Frame:")
print(df)

# 输出
=====

请输入您的提示 (输入 'exit' 退出) :
如何学习 python 数据分析编程，给出关键语法和学习案例'''
```

Online-softmax的进阶内容

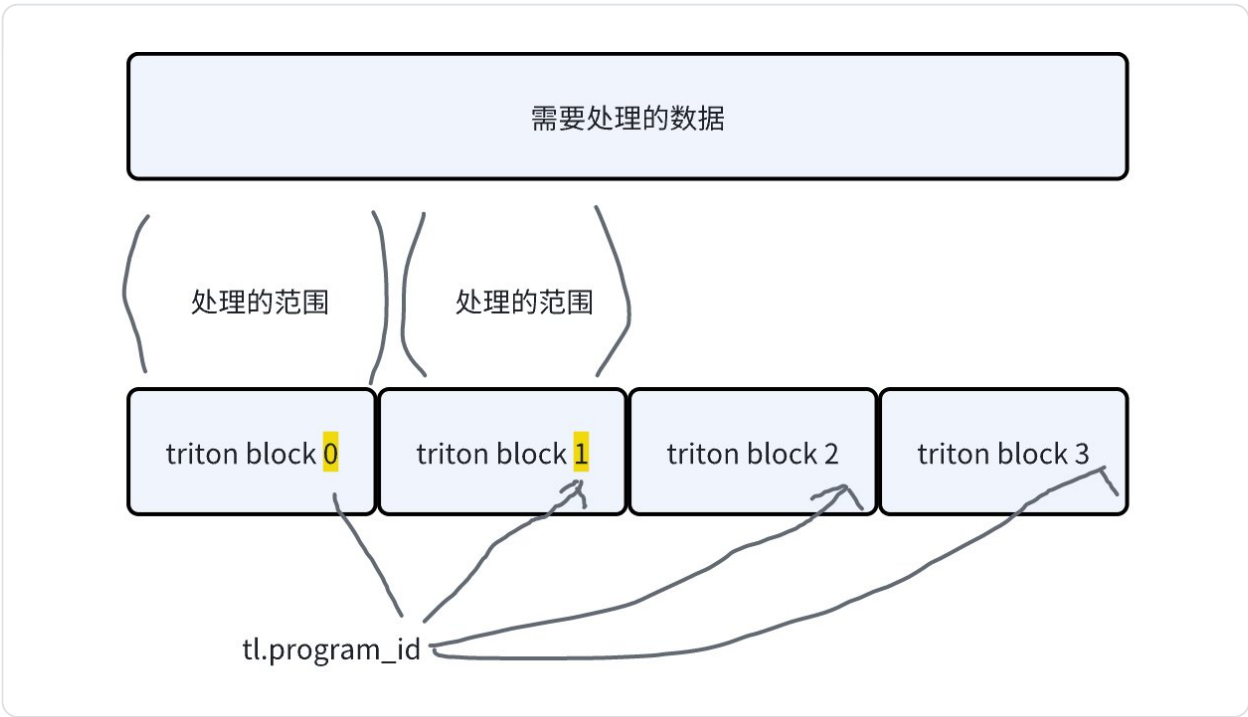
看完本节课程之后再去看

[第7课-《速通OpenAI Triton-从零自制轻量级大模型推理框架》-使用Triton实现FlashAttention](#)

我们上一节课为大家讲解了VecAdd，也就是向量加法算子，包括核函数的编写，启动的配置以及整体的流程。本节课在此基础上将进一步为大家讲解Softmax算子的编写。在此之前，我们会带大家回忆一下Triton核函数中的几个重要调用。

常用的 Triton 基础函数及其作用如下：

- `tl.load`：用于从由指针定义的内存位置加载数据。它的接口定义是 `triton.language.load(pointer, mask=None, other=None)`，首先 `pointer` 是你加载的数据指向的位置，`mask` 一般用于防止越界访问。比如 `pointer` 指向的是一组指针数组（常用于一次性加载多个数据），有可能数组中的某个指针超出了输入张量的范围，比如输入张量长为4，但是 `pointer` 等于 `[ptr+0, ptr+1, ptr+2, ptr+3, ptr+4]`，所以我们需要通过 `mask = [true, true, true, true, false]` 来指定有效的范围。
- `tl.store`：用于将张量的数据写入由指针定义的内存位置。具体的使用方法和 `tl.load` 是类似的，一般也需要指定输入的指针或指针输入，以及对应的有效范围掩膜 `Mask`。
- `tl.program_id(axis)`：返回当前块在指定轴上的 ID（编号），也就是在一个维度上有多个 triton block，我们现在要找到 triton block 的编号，以便于划分对应的数据处理范围。`axis` 是一个常量，因为在多个维度上都可以有 triton block，所以我们需要指定你想要查询的轴。
- 像下方的例子就是只有一个维度，但是对我们计算 softmax 已经是足够用的了，因为 softmax 算子本身也是一个一维算子。



- `tl.arange`：在半开区间 `[start, end)` 内返回连续值，用于生成从 0 开始的偏移量。举个例子，当 `start` 等于 0 且 `end` 等于 32 的时候，该区间就等于 `(0, 1, 2, 3, ..., 31)`。一般可以和指针位置一起用，用于形成一个连续的指针数组。比如 `ptr + tl.arange(0,12)` 那么就是指向 `ptr+0` 开始的 12 个元素的指针。

也就是用于一个指针数组，`[ptr, ptr+1, ptr+2, ..., ptr+11]`

Softmax的公式定义

Softmax 函数将输入向量的每个元素转换为一个概率分布。给定一个输入向量 $z = [z_1, z_2, \dots, z_n]$ ，Softmax 函数的公式如下：

$$\text{Softmax}(z_i) = \frac{e^{z_i} - z_{\max}}{\sum_{j=1}^n e^{z_j - z_{\max}}}$$

其中：

- $z_{\max} = \max(z_1, z_2, z_3, \dots, z_n)$
- z_i 是输入向量的第 i 个元素。

$$\text{Softmax}(z_i) = \frac{e^{z_i} - z_{\max}}{\sum_{j=1}^n e^{z_j - z_{\max}}}$$



大森林广播 2025年10月14日
分子的 $z_{\max}$ 在指数上

$$z_i = \frac{e}{\sum_{j=1}^n}$$

- $\sum_{j=1}^n e^{z_j}$ 是所有元素的指数和。

用Pytorch实现Softmax函数

为了深入理解Softmax公式的原理，并验证后续实现的Triton核函数的正确性，我们将首先使用PyTorch实现Softmax函数。这个PyTorch实现将作为基准参考，用于与后续的Triton实现进行对比验证。

代码块

```
1  import torch
2
3  def softmax(x):
4      x_exp = torch.exp(x - torch.max(x, dim=-1,
5      keepdim=True).values)
6      return x_exp / torch.sum(x_exp, dim=-1, keepdim=True)
7
8  if __name__ == '__main__':
9      input_tensor = torch.randn(1000, 512)
10     output_custom = softmax(input_tensor)
```

在上述代码中，softmax核函数的输入的维度是1000×512，我们对最后一维（即维度512）求softmax。其中，`torch.max(x, dim=-1)` 和 `torch.sum(x, dim=-1)` 依次对输入的最后维上求最大值和累计值。

用Triton实现Softmax函数

为了确保后续开发环境的一致性，请大家首先确认已正确安装Triton库。建议统一使用Triton 3.1.0以上版本，该版本与2.0系列存在显著差异。

代码块

```
1  $ pip list|grep triton
2  triton      3.1.0
```

让我们探讨如何利用分块策略来实现Softmax计算。已知输入矩阵的维度为M×N，其中M=1000表示行数，N=512表示列数。我们的设计方案是将1000行数据划分成K个块（Block），每个块负责处理约1000/K行的Softmax计算任务。

完整代码分析

下面的示意图是视频画的，请结合视频一起看，不要认为这是我乱涂乱画的。

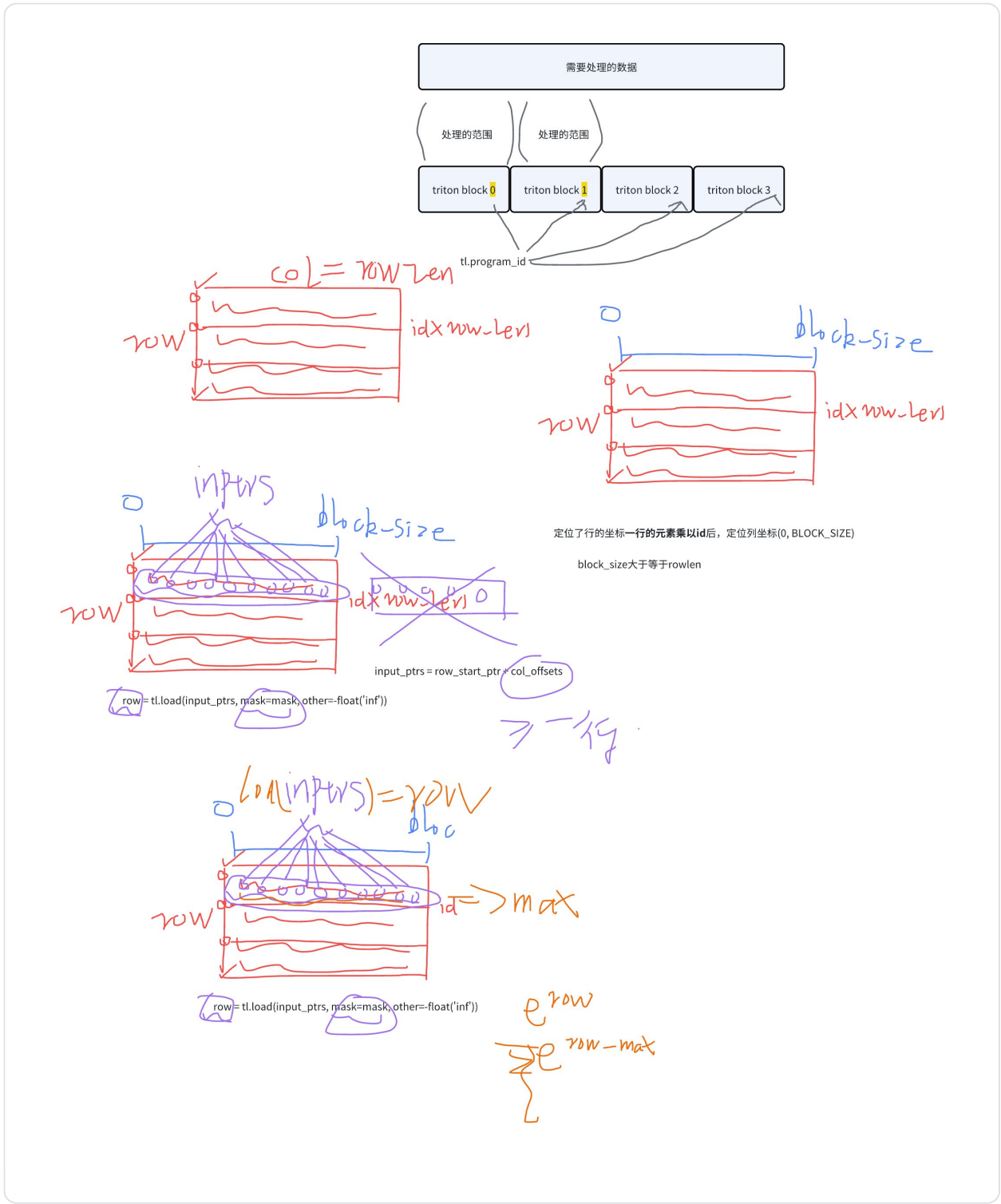
[画板]



ArdenX 3月6日 15:09

col=row_len?

row_len 不是一个 block 处理的行数吗？视频讲解也有问题。



我们将用多个triton block处理输入张量的多行，并独立在每行上求解softmax，所以首先我们需要确定每个triton block处理的行范围。

1. 确定每个Triton Block处理的行范围

- 行起始偏移：`row_start = triton.program_id(0) * row_len`，表示当前 Triton Block处理的行起始位置。因为我们的输入二维的，所以先定位到某一行的输
- 行起始指针：`row_start_ptr` 是基于 `row_start` 计算得到的，它指向当前行的起始位置（与 `row_start` 的区别在于，`row_start` 是偏移量，而 `row_start_ptr` 是具体的指针），相当于定位了我现在要处理的行数据。

2. 定位当前行的输入数据

加载一行不仅需要一行坐标，也需要对应的列坐标

- 将输入地址加上行偏移 `row_start`，得到 `row_start_ptr`，即当前行的起始指针。
- 再加上列偏移 `col_offsets`，得到 `input_ptrs`，即当前行的每个元素的具体地址。
- 使用 `tl.load` 从 `input_ptrs` 中加载当前行的元素，得到当前行当前列范围内的所有输入数据。

简单来说就是定位到行、列位置，行为`row_start`，列为`col_offsets`。

3. 在当前行上计算Softmax

- **求最大值**：对当前行的所有元素求最大值，用于后续的归一化计算（防止数值溢出）。
- **归一化**：将每个元素减去最大值后，计算指数并归一化，得到Softmax的结果。

4. 将Softmax结果写回输出

- 定位输出位置的方式与定位输入位置的方式相同：
 - 使用行偏移 `row_start` 和列偏移 `col_offsets`，计算输出指针 `output_ptrs`。
- 使用 `tl.store` 将计算后的Softmax结果写入到 `output_ptrs` 指向的位置。

代码块

```
1  @triton.jit
2  def softmax_kernel(output_ptr, input_ptr, input_row_stride,
3                     output_row_stride, n_rows, n_cols, BLOCK_SIZE: tl.constexpr):
4      row_len = 2
5      row_start = tl.program_id(0) * row_len
6      if row_start >= n_rows:
7          return
8      for row_idx in tl.range(row_start, row_start + row_len, 1):
9          # 定位一行的起点
10         row_start_ptr = input_ptr + row_idx * input_row_stride
11         col_offsets = tl.arange(0, BLOCK_SIZE)
12         # 表示某行中的所有元素下标
13         input_ptrs = row_start_ptr + col_offsets
14         mask = col_offsets < n_cols
15         # 加载一行中的所有元素
16         row = tl.load(input_ptrs, mask=mask, other=-float('inf'))
17
18         # 计算
19         row_minus_max = row - tl.max(row, axis=0)
20         numerator = tl.exp(row_minus_max)
21         denominator = tl.sum(numerator, axis=0)
22         softmax_output = numerator / denominator
23
24         # 为了写回定位一行的起点
25         output_row_start_ptr = output_ptr + row_idx *
26         output_row_stride
27         output_ptrs = output_row_start_ptr + col_offsets
28         # 写回
29         tl.store(output_ptrs, softmax_output, mask=mask)
```

Triton中的分块计算策略

我们刚才说到一个triton block可以一行softmax的运算，但是也可以负责多行

我们知道内核在 **triton** 中是并行执行的，每个内核负责处理的数据范围不一样，内核执行的个数跟块（Block）数有关，多内核并行实际就是多块并行，即多个块可以在不同的多处理器（SMs）上同时运行，每个块内的线程也在其所属的 SM 上并行执行。为了更好的理解有多少个块（内核）并行执行，我们需要知道各自执行的数据范围是多少。

我们在下图中展现了这一例子，**每个块需要计算两个行**。我们在上节课中讲过，Triton的并行以块作为单位，而并不在块的基础上再细分出线程，**第一个块block1将负责第1, 2两行的计算**，第二个块block2将负责第3, 4两行的计算，第三个块block3将负责第5, 6两行的计算，以此类推。



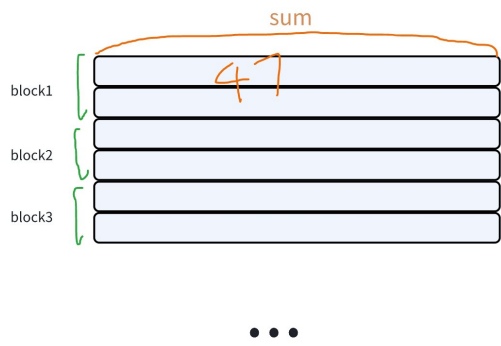
基于上述设计，我们实现了部分Triton代码。其中，`row_len` 定义了每个block所处理的行数；`row_start` 则标识了当前block处理的起始行号，用于定位其在全局数据中的行位置。那么我们的for循环就是在处理一个block中对应的两行。

和上面的例子唯一的区别就是一个block会处理多行。

代码块

```
1  @triton.jit
2  def softmax_kernel(output_ptr, input_ptr, input_row_stride,
3                     output_row_stride, n_rows, n_cols, BLOCK_SIZE: tl.constexpr):
4      row_len = 2
5      row_start = tl.program_id(0) * row_len
6      if row_start >= n_rows:
7          return
8      for row_idx in tl.range(row_start, row_start + row_len, 1):
```

Block对每行的规约计算



- 由于Softmax函数的计算特性，我们需要对输入张量中的每一行计算其最大值。
- 1. 在Triton框架中，已经提供了用于Kernel中的规约操作函数 `tl.max`（即 `triton.language.max`），可以帮助我们高效地完成这一任务。
 - 2. 具体来说，为了实现行最大值的计算，我们首先需要加载该行的所有数据。可以通过 `tl.arange(0, 列数)` 结合每行的起始地址，来表示该行中各个元素的内存地址偏移
 - 3. 接着，使用 `tl.load` 函数从内存中读取这些数据，然后通过 `tl.max` 对该行的数据进行最大值规约运算。这一过程确保了我们在处理每一行数据时能够高效地获取最大值，为后续的Softmax计算打下坚实的基础。

triton.language.max

```
triton.language.max(input, axis=None, return_indices=False, return_indices_tie_break_left=True, keep_dims=False)
```

Returns the maximum of all elements in the `input` tensor along the provided `axis`

- Parameters:
- `input` (*Tensor*) – the input values
 - `axis` (*int*) – the dimension along which the reduction should be done. If `None`, reduce all dimensions
 - `keep_dims` (*bool*) – if true, keep the reduced dimensions with length 1
 - `return_indices` (*bool*) – if true, return index corresponding to the maximum value
 - `return_indices_tie_break_left` (*bool*) – if true, in case of a tie (i.e., multiple elements have the same maximum value), return the left-most index for values that aren't NaN

This function can also be called as a member function on `tensor`, as `x.max(...)` instead of `max(x, ...)`.

我们再根据以上的分析写出对应部分的Triton代码，`row_start_ptr = input_ptr + row_idx * input_row_stride` 表示每行的起始位置，`col_offsets` 表示当前行中所有列的偏移量。`mask` 表示掩码，防止每行的数据访问超出列数限制。

综上所述，`row_start_ptr + col_offsets` 就表示第 `row_start` 行中的所有元素偏移，随后我们再用 `tl.load` 加载一行的所有元素，记作 `row`。

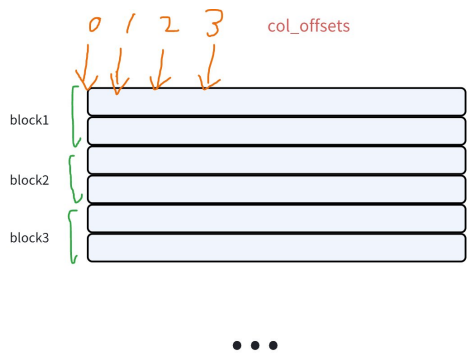
另外，这里每行的范围属于 `(0, BLOCK_SIZE)`，其中`BLOCK_SIZE`是大于列数的数值，同时也是一个2的幂次方。所以，在后续计算中，我们需要用`mask`来屏蔽部分超出一列范围的数值。

```
代码块

1  for row_idx in tl.range(row_start, row_start + row_len, 1):
2      row_start_ptr = input_ptr + row_idx * input_row_stride
3      col_offsets = tl.arange(0, BLOCK_SIZE)
4      input_ptrs = row_start_ptr + col_offsets
5
6      mask = col_offsets < n_cols
7      row = tl.load(input_ptrs, mask=mask, other=-float('inf'))
```

下图中每一行的对应位置都可以用偏移 `col_offsets` 来表示，`col_offsets`就是我们在前文中说过的`tl.range()`表示的一个范围，和指针地址相加可以表示一个指针数组，用于加载一个数据范围，我们随后还将对一行的数据，也就是 `row` 变量（维度为`1×512`）计算最大值。

1. `row_start_ptr`：当前行在内存中的起始地址；
2. `col_offsets`：表示当前行中各个列位置的偏移量（以元素个数为单位）；
3. `input_ptrs`：由 `row_start_ptr` 和 `col_offsets` 相加得到，表示当前行中需要加载的各列元素的内存地址集合



以下是对维度为 1×512 的张量 `row` 求它的最大值 `tl.max(row, axis=0)`，并将一行中的每个值都减去这个最大值得到 `row_minus_max`，最后再将 `row_minus_max`（维度为 1×512 ）求指数后再求和得到 `denominator`。

此处的`row`就是上文中说过的被加载的一行的数据，我们在这里对它求最大值（行最大值）。

代码块

```
1 row_minus_max = row - tl.max(row, axis=0)
2
3 numerator = tl.exp(row_minus_max)
4 denominator = tl.sum(numerator, axis=0)
5 softmax_output = numerator / denominator
```

Triton核函数的启动配置

我们还记得，我将使用一个块(Block) 计算输入张量中的两行，所以在外部循环中我们指定当前块负责计算的是第`row_start`行和第`row_start + 1`行。

代码块

```
1 for row_idx in tl.range(row_start, row_start + row_len, 1):
```

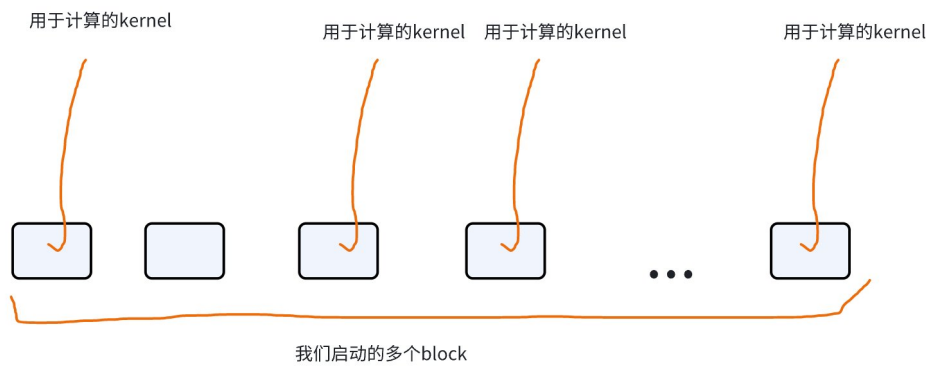
那么在外部，我们需要配置的块数等于行数除以2。如果一个triton block处理N个块，`grid`就是`n_rows // N`

代码块

```
1 grid = lambda meta: (n_rows // 2,)
```

在 Triton 的实现中，我们通常需要根据输入张量的维度来配置执行网格（grid）的大小。在本例中，为了处理 Softmax 输入张量的每一行，我们将每个 Triton block 分配处理两行（即 `N = 2` 行），因此所需的 block 数量为总行数除以 2。

`grid` 参数定义了一个 `lambda` 函数，用于动态计算网格的尺寸，而不是指定一个具体的固定数值。



结果对比

实现了Pytorch版本的Sofmax函数和Triton版本的，我们就需要编写测试代码来验证两者结果的一致性。

代码块

```
1  input_tensor = torch.randn(1000, 512, device='cuda') # 1000x512 的
   随机张量
2  output_tensor = torch.empty_like(input_tensor) # 用于存储输出的张量
3
4  # 2. 定义 kernel 的网格和块大小
5  n_rows, n_cols = input_tensor.shape
6  BLOCK_SIZE = triton.next_power_of_2(n_cols) # 计算大于 n_cols 的下一
   个 2 的幂
7  num_stages = 3 # 可以根据硬件调整
8
9  # 3. 调用 kernel
10 grid = lambda meta: (triton.cdiv(n_rows, 2),) # 网格大小为行数
11 softmax_kernel[grid](
12     output_tensor, input_tensor,
13     input_tensor.stride(0), output_tensor.stride(0), # 行步长
14     n_rows, n_cols,
15     BLOCK_SIZE=BLOCK_SIZE,
16 )
17
18 # 4. 验证结果
19 # 使用 PyTorch 的 softmax 作为参考
20 expected_output = torch.softmax(input_tensor, dim=1)
21 print("Triton Softmax 和 PyTorch Softmax 是否接近:",
      torch.allclose(output_tensor, expected_output, atol=1e-6))
```